

Weiterentwicklung eines selbstfahrenden Fahrzeugs mit Lidar und anderen Sensoren

STUDIENARBEIT

des Studienganges Informatik
an der Dualen Hochschule Baden-Württemberg Ravensburg Campus Friedrichshafen

von

Elena Schwarzbach, 3830156, TIK23

Sonia Sinaci, 8366601, TIK23

Scott Jonathan Hebach, 8958697, TIT23

Marius Maurer, 9339665, TIT23

7. Juli 2026

Bearbeitungszeitraum: x Wochen

Gutachter der Dualen Hochschule: Titel Claudia Zinser

Eidesstattliche Erklärung

gemäß Ziffer 1.1.13 der Anlage 1 zu §§ 3, 4 und 5 der Studien- und Prüfungsordnung für die Bachelorstudiengänge im Studienbereich Technik der Dualen Hochschule Baden-Württemberg vom 29.09.2017.

Ich versichere hiermit, dass ich meine Bachelorarbeit (bzw. Projektarbeit oder Studienarbeit bzw. Hausarbeit) mit dem Thema:

Weiterentwicklung eines selbstfahrenden Fahrzeugs mit Lidar und anderen Sensoren

selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe. Ich versichere zudem, dass die eingereichte elektronische Fassung mit der gedruckten Fassung übereinstimmt.

Friedrichshafen, 7. Juli 2026

Elena Schwarzbach

Sonia Sinaci

Scott Jonathan Hebach

Marius Maurer

Genderhinweis

In der vorliegenden Arbeit wird aus Gründen der sprachlichen Ökonomie und zur Sicherung eines einheitlichen Schriftbildes auf eine durchgängige geschlechts-spezifische Differenzierung verzichtet. Sämtliche Personen-bezeichnungen im generischen Maskulinum verstehen sich als geschlechtsneutral und schließen alle Geschlechter gleichermaßen ein. Diese Entscheidung basiert rein auf redaktionellen Überlegungen und spiegelt keine Geringschätzung gegenüber weiblichen oder nicht-binären Personen wider. Die Autoren sind sich der Wirkung von Sprache auf die Wahrnehmung bewusst und verfolgen mit dieser Wahl das Ziel, die Komplexität des Textes zugunsten der inhaltlichen Klarheit zu reduzieren.

Inhaltsverzeichnis

Abkürzungsverzeichnis	I
Abbildungsverzeichnis	I
Tabellenverzeichnis	II
1 Einleitung	1
2 Ist-Zustand	2
2.1 App-Steuerung (Elena)	2
2.2 Funktionierend	2
2.3 Probleme	2
3 Technikkorridor	3
3.1 Hardwarevergleich Raspberry Pi 4 und Raspberry Pi 5	3
3.2 Vergleich unterschiedlicher Betriebssysteme für Raspberry Pi	5
3.3 Softwarearchitektur	6
3.4 Betrachtete Systemarchitekturen	8
3.5 Kommunikationskonzept	9
3.6 Entscheidung und Begründung	10
3.7 Fazit	11
4 Autoubau	13
5 Autonomes Fahren	14
5.1 Ausweichkurven	14
5.2 Herausforderungen	14
6 Implementierung Autonomes Fahren	15
6.1 Hinderniserkennung (Elena)	15
6.2 Gauß-basierte Hindernisvermeidung (Elena)	16
6.3 Optimierung der Spurplanung (Elena)	17
6.4 Priorisierung der Fahrbefehle (Elena)	18
6.5 Ideen	18

6.6 Herausforderungen	18
7 Aktualisierung der Applikation (Elena)	19
8 Ergebnisse	22
8.1 Offene Herausforderungen	22
8.1.1 Grenzen der Hindernisvermeidung (Elena)	22
8.2 Positives	23
9 Fazit und Ausblick	24
Literaturverzeichnis	V

Abbildungsverzeichnis

1	Klassendiagramm: Ausweichen	19
2	Klassendiagramm: Ausweichen	20

Tabellenverzeichnis

1	Direkter Hardwarevergleich zwischen Raspberry Pi 4 und Raspberry Pi 5	4
2	Vergleich relevanter Betriebssystemoptionen	6
3	Vergleich zwischen nativer ROS2-Installation und Docker-basierter ROS2-Architektur	8
4	Bewertung betrachteter Systemarchitekturen	9

1 Einleitung

2 Ist-Zustand

2.1 App-Steuerung (Elena)

Die App besitzt bereits eine theoretische Steuerung über die Bewegungssensoren des Smartphones. Dafür wird in `gyroscope_data.dart` die Bibliothek `motion_sensors` verwendet. Das Widget `getGyroscope` liest bereits die Werte für Pitch, Roll und Yaw aus. Sobald neue Sensordaten eintreffen, werden Pitch und Roll auf den Bereich von -1.0 bis 1.0 begrenzt. Anschließend werden diese Werte direkt als Steuerbefehle über den `WebSocketManager` an das Fahrzeug gesendet. Dabei soll der Roll-Wert die Geschwindigkeit steuern und der Pitch-Wert für den Lenkwinkel verwendet werden. Der Yaw-Wert wird für die Steuerung nicht verwendet. Zusätzlich zeigt die App die aktuellen Sensorwerte für Pitch, Roll und Yaw auf der Oberfläche an.

In diesem Stand ist die Funktion zwar im Code angelegt, funktioniert praktisch aber noch nicht. Ein zentrales Problem liegt darin, dass die App von `motion_sensors.orientation` abhängig ist. Das Paket `motion_sensors` ist jedoch veraltet und liefert keine Werte. Außerdem werden die Sensorwerte direkt weitergegeben, ohne sie vorher zu filtern oder zu glätten. Dadurch könnten bereits kleinste Bewegungen oder Messungenauigkeiten die Steuerung beeinflussen.

Weitere Probleme sind, dass es keine Deadzone für unbeabsichtigte kleine Bewegungen gibt und keine Möglichkeit vorgesehen ist, die Empfindlichkeit der Steuerung anzupassen. Auch eine einfache Korrektur falsch herum reagierender Achsen ist nicht vorbereitet.

2.2 Funktionierend

2.3 Probleme

3 Technikkorridor

Der Technikkorridor beschreibt den technisch sinnvollen Lösungsraum für die Realisierung eines autonomen Fahrzeugs auf Raspberry-Pi-Basis. Ziel ist es, aus den verfügbaren Hardware-, Betriebssystem- und Softwareoptionen eine Plattform abzuleiten, die für echtzeitnahe Sensorverarbeitung, Kamerabildverarbeitung, Batteriebetrieb, Wartbarkeit und Erweiterbarkeit geeignet ist.

Im Mittelpunkt steht dabei die Frage, welche Kombination aus Einplatinencomputer, Betriebssystem, Robotik-Middleware und optionaler Containerisierung für ein Studien- oder Forschungsprojekt den besten Kompromiss aus Leistungsfähigkeit, Ressourcenbedarf und Entwicklungsaufwand bietet. Für autonome Fahrzeuge sind insbesondere parallele Sensordatenströme, Bildverarbeitung, stabile Kommunikation zwischen Softwarekomponenten sowie eine robuste und wartbare Systemstruktur relevant.

Der Raspberry Pi 4 und der Raspberry Pi 5 stellen dabei zwei naheliegende Hardwareoptionen dar. Während der Raspberry Pi 4 weiterhin für bestehende oder kostenkritische Anwendungen geeignet ist, bietet der Raspberry Pi 5 durch seine höhere CPU-Leistung, bessere I/O-Anbindung und Erweiterbarkeit über PCIe eine deutlich leistungsfähigere Grundlage für Robotik- und autonome Fahrzeugsysteme [zotero-item-143] [zotero-item-144]. Ergänzend beeinflussen Betriebssystemwahl, Softwarearchitektur und Kommunikationskonzept maßgeblich die Praxistauglichkeit des Gesamtsystems.

3.1 Hardwarevergleich Raspberry Pi 4 und Raspberry Pi 5

Der Hardwarevergleich dient dazu, die Eignung des Raspberry Pi 4 und des Raspberry Pi 5 für ein autonomes Fahrzeug einzuordnen. Beide Plattformen besitzen eine ähnliche Grundidee als kompakte Einplatinencomputer, unterscheiden sich jedoch deutlich hinsichtlich Rechenleistung, Speicheranbindung, I/O-Architektur und Erweiterbarkeit.

Der Raspberry Pi 4 basiert auf einer Cortex-A72-Plattform und ist mit bis zu 8 GB LPDDR4-Arbeitsspeicher verfügbar. Er bietet zwei USB-3.0- und zwei USB-2.0-Anschlüsse, Gigabit-Ethernet sowie zwei Micro-HDMI-Ausgänge für Dual-Display-Betrieb bis 4K. Damit ist er für allgemeine IoT-, Embedded- und Server-Anwendungen weiterhin brauchbar. Für rechenintensive Aufgaben wie parallele Sensorverarbeitung, Bildauswertung oder mehrere gleichzeitig laufende Robotik-Komponenten stößt er jedoch

früher an Leistungsgrenzen [zotero-item-143].

Der Raspberry Pi 5 stellt demgegenüber die technisch modernere Plattform dar. Er verwendet eine Cortex-A76-basierte CPU und erreicht nach Herstellerangaben eine etwa zwei- bis dreifache CPU-Leistungssteigerung gegenüber dem Raspberry Pi 4 [zotero-item-144]. Hinzu kommen LPDDR4X-Arbeitsspeicher, der neue RP1-I/O-Controller, eine PCIe-2.0-x1-Schnittstelle, eine integrierte Echtzeituhr sowie ein Power-Button auf dem Board [zotero-item-144]. Besonders relevant ist die PCIe-Schnittstelle, da sie über entsprechende Adapter den Einsatz von NVMe-SSDs ermöglicht. Dadurch können Logdaten, Kartenmaterial, Container-Images oder Build-Artefakte deutlich performanter gespeichert und gelesen werden.

Auch externe Benchmarks bestätigen den deutlichen Leistungssprung. In Geekbench-6-Messungen erreicht der Raspberry Pi 5 im Single-Core-Test etwa 764 bis 774 Punkte, während der Raspberry Pi 4 bei ungefähr 340 Punkten liegt. Dies entspricht etwa dem 2,4-fachen Leistungsniveau [zotero-item-146]. Zusätzlich werden dem Raspberry Pi 5 im Vergleich zum Raspberry Pi 4 eine höhere Speicher- und SD-Karten-Performance sowie eine verbesserte Kamera- und I/O-Anbindung zugeschrieben [zotero-item-147].

Tabelle 1: Direkter Hardwarevergleich zwischen Raspberry Pi 4 und Raspberry Pi 5

Merkmal	Raspberry Pi 4	Raspberry Pi 5
CPU	Cortex-A72	Cortex-A76
Arbeitsspeicher	LPDDR4, bis zu 8 GB	LPDDR4X
PCIe	nicht vorhanden	PCIe 2.0 x1
I/O-Architektur	klassisch integriert	RP1-I/O-Controller
USB-Bandbreite	geringer	höher
Kameraanbindung	geringer	verbessert
RTC	nicht hervorgehoben	auf dem Board vorhanden
Power-Button	nicht hervorgehoben	auf dem Board vorhanden

Für ein autonomes Fahrzeug ist der Raspberry Pi 5 vor allem aufgrund seiner höheren CPU-Reserven und verbesserten I/O-Fähigkeiten geeigneter. Sensorfusion, Kamerabildverarbeitung und mehrere parallel laufende Softwarekomponenten profitieren stark von dieser zusätzlichen Leistung. Der Raspberry Pi 4 bleibt hingegen eine sinnvolle Option für bestehende Projekte oder kostenkritische Anwendungen, wenn die Anforderungen an Bildverarbeitung, Datenrate und Erweiterbarkeit begrenzt bleiben.

Bei der Energieversorgung ist zwischen Netzteilanforderung und tatsächlicher Leistungsaufnahme zu unterscheiden. Für den Raspberry Pi 5 nennt der Hersteller eine

Stromversorgung über USB-C mit 5 V und 5 A über USB Power Delivery [zotero-item-144]. Für den Raspberry Pi 4 Model B werden 5 V und 3 A über USB-C oder GPIO angegeben; ein gutes 2,5-A-Netzteil kann genügen, wenn die USB-Peripherie weniger als 500 mA benötigt [zotero-item-150]. Diese Angaben sind jedoch Versorgungsanforderungen und dürfen nicht unmittelbar als durchschnittlicher Energieverbrauch interpretiert werden.

3.2 Vergleich unterschiedlicher Betriebssysteme für Raspberry Pi

Die Wahl des Betriebssystems beeinflusst Ressourcenverbrauch, Hardwareintegration, ROS2-Kompatibilität, Wartbarkeit und Entwicklungsaufwand. Für ein autonomes Fahrzeug sind insbesondere schlanke Systeme mit guter Treiberunterstützung und stabiler Langzeitnutzung relevant. Betrachtet werden Raspberry Pi OS Lite, Ubuntu Server, Ubuntu Desktop und DietPi. Zusätzlich wird die Bedeutung von ARM64 für ROS2 berücksichtigt.

Raspberry Pi OS Lite ist das offiziell naheliegende Betriebssystem für viele Raspberry-Pi-Anwendungsfälle. Es verzichtet auf eine Desktop-Umgebung und eignet sich daher besonders für Headless-Server, Embedded-Anwendungen und batteriebetriebene Systeme. Der geringe Ressourcenbedarf reduziert RAM-Verbrauch, Hintergrundlast und tendenziell auch den Energiebedarf. Raspberry Pi OS Lite 64 Bit wird für Raspberry Pi 3, 4 und 5 empfohlen und bietet eine sehr gute Hardwareintegration [zotero-item-149] [zotero-item-151].

Ubuntu Server ist vor allem dann attraktiv, wenn das Projekt stark am Ubuntu- und ROS-Ökosystem ausgerichtet ist. Viele ROS2-Installationsanleitungen, Pakete und Entwicklungsabläufe beziehen sich direkt auf Ubuntu. Dadurch kann Ubuntu Server den Entwicklungsaufwand reduzieren, wenn bereits Linux-Server-Know-how vorhanden ist oder wenn professionelle DevOps-nahe Workflows genutzt werden sollen [zotero-item-148]. Ubuntu nennt als Systemanforderungen für Server-Installationen unter anderem einen Mindestbedarf an Arbeitsspeicher und Massenspeicher, wobei für produktive Robotikprojekte ausreichend schneller Speicher eingeplant werden sollte [zotero-item-153].

Ubuntu Desktop eignet sich besonders für Entwicklung, Simulation und Debugging direkt auf dem Raspberry Pi. Werkzeuge wie RViz und Gazebo können lokal genutzt werden, was bei der Entwicklung und Analyse von ROS2-Systemen hilfreich ist. Gleich-

zeitig benötigt Ubuntu Desktop deutlich mehr RAM, Speicherplatz und Rechenleistung als ein Headless-System. Für den produktiven Fahrzeugeinsatz ist es daher weniger geeignet, kann aber als Entwicklungsplattform sinnvoll sein [zotero-item-154].

DietPi ist eine besonders schlanke Alternative für ressourcenschonende Embedded- oder Server-Setups. Es kann interessant sein, wenn minimale Systemlast wichtiger ist als maximale offizielle Raspberry-Pi-Hardwareintegration. Da in der übergebenen BibTeX-Datei jedoch kein eigener DietPi-Eintrag enthalten ist, sollte für eine endgültige wissenschaftliche Fassung noch ein passender BibTeX-Eintrag ergänzt werden.

Für ROS2 ist ARM64 besonders relevant, da diese Architektur im Raspberry-Pi-Kontext eine wichtige Rolle spielt. Eine direkte native ROS2-Installation ist insbesondere auf Ubuntu naheliegend. Raspberry Pi OS 64 Bit kann dagegen in Kombination mit Docker genutzt werden, um Ubuntu-basierte ROS2-Container auszuführen [zotero-item-156].

Tabelle 2: Vergleich relevanter Betriebssystemoptionen

Kriterium	Pi OS Lite	Ubuntu Server	Ubuntu Desktop	DietPi
RAM-Bedarf	sehr gering	mittel	hoch	sehr gering
Strombedarf	gering	mittel	hoch	gering
ROS2-Nähe	gut	sehr gut	sehr gut	abhängig vom Se
Hardwareintegration	sehr gut	gut	gut	gut bis mittel
Entwicklung am Gerät	eingeschränkt	eingeschränkt	sehr gut	eingeschränkt
SSD/NVMe-Einsatz	optional sinnvoll	sinnvoll	sehr sinnvoll	optional

Für ein autonomes Fahrzeug mit Batteriebetrieb und begrenzten Ressourcen ist Raspberry Pi OS Lite 64 Bit besonders geeignet. Es bietet eine gute Kombination aus geringer Systemlast und sehr guter Hardwareunterstützung. Ubuntu Server bleibt eine starke Alternative, wenn ROS2-Kompatibilität, vorhandenes Ubuntu-Know-how oder spätere Skalierung im Vordergrund stehen. Ubuntu Desktop sollte primär als Entwicklungs- und Debugging-Umgebung betrachtet werden.

3.3 Softwarearchitektur

Die Softwarearchitektur legt fest, wie Sensorik, Aktorik, Wahrnehmung, Steuerung und Diagnose logisch strukturiert und miteinander verbunden werden. Für autonome Fahrzeuge bietet sich eine modulare Architektur an, bei der einzelne Funktionen in getrennten Softwarekomponenten ausgeführt werden. Dadurch können Sensorverarbeitung,

Lokalisierung, Fahrentscheidung und Motorsteuerung unabhängig entwickelt, getestet und gewartet werden.

ROS2 eignet sich als zentrale Middleware für ein solches System. Es unterstützt die modulare Kommunikation zwischen Knoten und ist in Robotik, autonomen Fahrzeugsystemen und Forschungsprojekten weit verbreitet. Sensoren, Aktoren, Steuerungslogik und Diagnosefunktionen können in ROS2 als getrennte Nodes realisiert werden. Die Kommunikation erfolgt über Topics, Services und Actions, wodurch eine klare Trennung zwischen Datenströmen, Steuerbefehlen und länger laufenden Aktionen möglich wird [**zotero-item-156**].

Docker kann ergänzend eingesetzt werden, um Softwareabhängigkeiten und Laufzeitumgebungen in Container zu kapseln. Dies verbessert Reproduzierbarkeit, Wartbarkeit und Deployment, insbesondere wenn mehrere Entwickler beteiligt sind oder mehrere Softwarekomponenten unabhängig voneinander aktualisiert werden müssen. Für kleinere Einzelentwicklerprojekte kann Docker jedoch zusätzliche Komplexität verursachen.

Wissenschaftliche Untersuchungen zeigen, dass Linux-Container bei CPU- und Speicheroperationen häufig nahezu native Performance erreichen können, während relevanter Overhead vor allem bei I/O- und Netzwerkpfaden entstehen kann [**instituteofelectricalandele**]. Spezifische Untersuchungen auf verschiedenen Raspberry-Pi-Modellen zeigen zudem, dass Docker grundsätzlich auch auf ressourcenbegrenzten Raspberry-Pi-Systemen einsetzbar ist. Die praktische Skalierbarkeit hängt jedoch stark vom verfügbaren Arbeitsspeicher und von der jeweiligen Container-Workload ab [**gamePerformanceEvaluationDock**].

Der Ressourcenverbrauch von Containern sollte projektbezogen gemessen werden. Docker stellt hierfür unter anderem Metriken zu CPU-Auslastung, Speicherverbrauch, Netzwerk-I/O und Block-I/O bereit [**zotero-item-163**]. Außerdem verursacht Docker keinen festen, pauschalen RAM-Overhead pro Container. Der tatsächliche Ressourcenbedarf hängt vom Container-Image, von der laufenden Anwendung, von der Anzahl paralleler Container und von gesetzten Ressourcenlimits ab [**dockerinc.RuntimeMetrics**].

Für ein Studienprojekt oder ein Einzelentwicklerprojekt überwiegen häufig die Vorteile einer nativen ROS2-Installation: geringere Komplexität, direkter Hardwarezugriff und weniger Ressourcenbedarf. Docker lohnt sich insbesondere dann, wenn reproduzierbare Deployments, klare Trennung mehrerer Komponenten, Teamarbeit oder CI/CD-Prozesse relevant sind.

Tabelle 3: Vergleich zwischen nativer ROS2-Installation und Docker-basierter ROS2-Architektur

Kriterium	ROS2 nativ	Docker + ROS2
Energieeffizienz	besser	tendenziell schlechter
RAM-Verbrauch	geringer	höher bzw. workloadequabhängig
Deployment	aufwendiger	einfacher reproduzierbar
Wartbarkeit	geringer	höher
Teamarbeit	weniger komfortabel	besser geeignet
Reproduzierbarkeit	geringer	höher
Hardwarezugriff	direkter	konfigurationsabhängig
Komplexität	geringer	höher

3.4 Betrachtete Systemarchitekturen

Aus der Kombination von Hardware, Betriebssystem, ROS2 und optionaler Containerisierung ergeben sich mehrere technisch sinnvolle Systemarchitekturen. Diese unterscheiden sich insbesondere hinsichtlich Ressourcenbedarf, Wartbarkeit, Entwicklungsaufwand und Eignung für den produktiven Fahrzeugeinsatz.

Die erste Variante ist Raspberry Pi OS Lite 64 Bit mit nativer ROS2-Installation. Diese Architektur ist besonders ressourcenschonend und hardwarenah. Sie eignet sich für Einzelentwicklungen, Studienprojekte und batteriebetriebene Fahrzeuge, bei denen geringe Systemlast und einfache Wartung wichtiger sind als vollständig containerisierte Deployments. Der direkte Hardwarezugriff kann insbesondere bei GPIO, Kamera, Sensorik und Aktorik vorteilhaft sein.

Die zweite Variante ist Raspberry Pi OS Lite 64 Bit mit Docker und ROS2. Sie kombiniert die gute Raspberry-Pi-Hardwareintegration mit reproduzierbaren Softwareumgebungen. Diese Architektur ist sinnvoll, wenn mehrere ROS2-Komponenten sauber voneinander getrennt werden sollen oder wenn ein konsistentes Deployment auf mehreren Geräten erforderlich ist. Der Nachteil liegt im höheren Administrationsaufwand und im zusätzlichen Ressourcenbedarf.

Die dritte Variante ist Ubuntu Server mit nativer ROS2-Installation. Sie ist besonders geeignet, wenn das Projekt stark am Ubuntu- und ROS2-Ökosystem orientiert ist. Viele ROS2-Anleitungen und Pakete sind für Ubuntu ausgelegt, wodurch die Einrichtung erleichtert werden kann [zotero-item-156]. Gegenüber Raspberry Pi OS Lite ist jedoch mit einer höheren Systemlast zu rechnen.

Die vierte Variante ist Ubuntu Server mit Docker und ROS2. Diese Architektur bietet

hohe Modularität und Reproduzierbarkeit. Sie ist besonders für Teamprojekte, CI/CD-Prozesse und verteilte Softwarekomponenten geeignet. Gleichzeitig ist sie die komplexeste der betrachteten produktiven Varianten und benötigt sorgfältige Ressourcenüberwachung, insbesondere auf einem Raspberry Pi [gamePerformanceEvaluationDocker2023] [zotero-item-163].

Die fünfte Variante ist Ubuntu Desktop mit ROS2. Sie eignet sich vor allem als Entwicklungsplattform für grafische Werkzeuge wie RViz oder Gazebo. Für den produktiven Fahrzeugeinsatz ist sie wegen des höheren RAM-, Speicher- und Energiebedarfs weniger attraktiv [zotero-item-154].

Tabelle 4: Bewertung betrachteter Systemarchitekturen

Architektur	Stärken	Einschränkungen
Pi OS Lite + ROS2	effizient, hardwarenah	weniger reproduzierbares Deployment
Pi OS Lite + Docker + ROS2	wartbar, reproduzierbar	mehr Komplexität und Ressourcenbedarf
Ubuntu Server + ROS2	gute ROS2-Nähe	höhere Systemlast
Ubuntu Server + Docker + ROS2	modular, teamfähig	komplexeste Variante
Ubuntu Desktop + ROS2	gut für Entwicklung	ungeeignet für ressourcenarmen Betrieb

3.5 Kommunikationskonzept

Das Kommunikationskonzept beschreibt, wie auf das Fahrzeug zugegriffen wird und wie Steuerung, Diagnose und Wartung erfolgen. Für ein autonomes Fahrzeug ist eine robuste und möglichst unabhängige Kommunikationslösung wichtig, da das System auch außerhalb vorhandener Netzwerkinfrastruktur zuverlässig erreichbar bleiben soll.

Ein lokaler WLAN-Access-Point auf dem Raspberry Pi stellt hierfür eine geeignete Lösung dar. Das Fahrzeug baut dabei ein eigenes WLAN auf, mit dem sich ein Entwicklungsrechner, Tablet oder Smartphone verbinden kann. Dadurch sind Steuerung, Parametrierung, Diagnose und Logzugriff auch dann möglich, wenn kein externes Netzwerk vorhanden ist. Diese Betriebsart ist besonders praktisch für Tests in Laborumgebungen, auf Teststrecken oder in Bereichen ohne zuverlässige WLAN-Infrastruktur.

Alternativ kann der Raspberry Pi als WLAN-Client in ein vorhandenes Netzwerk eingebunden werden. Dies ist sinnvoll, wenn das Fahrzeug dauerhaft in einer Labor- oder Campusinfrastruktur betrieben wird. Der Vorteil liegt in der einfachen Integration in bestehende Netzwerke. Der Nachteil besteht darin, dass die Erreichbarkeit von der vorhandenen Infrastruktur abhängt.

Ethernet bietet eine robuste und latenzarme Verbindung, ist jedoch kabelgebunden. Es eignet sich daher besonders für Laborbetrieb, stationäre Tests, Einrichtung und Debugging, aber weniger für den mobilen Fahrzeugeinsatz. LTE oder 5G ermöglichen entfernte Einsätze und Fernzugriff, erhöhen jedoch Kosten, Energiebedarf und Systemkomplexität. Für ROS2-Systeme ist außerdem zu beachten, dass die zugrunde liegende DDS-Kommunikation netzwerkabhängig konfiguriert werden muss, insbesondere wenn mehrere Systeme über bestehende Infrastruktur miteinander kommunizieren.

Für das betrachtete autonome Fahrzeug ist daher ein lokaler WLAN-Access-Point als primäres Kommunikationskonzept sinnvoll. Ergänzend kann Ethernet für Einrichtung und Debugging vorgesehen werden. WLAN-Client-Betrieb oder Mobilfunk können optional ergänzt werden, wenn der konkrete Anwendungsfall dies erfordert.

3.6 Entscheidung und Begründung

Auf Basis des Hardwarevergleichs, der Betriebssystembewertung und der betrachteten Softwarearchitekturen wird folgende Zielplattform empfohlen:

- Raspberry Pi 5 als Hardwareplattform,
- Raspberry Pi OS Lite 64 Bit als Betriebssystem,
- ROS2 als zentrale Robotik-Middleware,
- Docker optional, abhängig von Teamgröße, Deployment-Anforderungen und Projektkomplexität,
- lokaler WLAN-Access-Point als primäres Kommunikationskonzept.

Der Raspberry Pi 5 wird gegenüber dem Raspberry Pi 4 bevorzugt, da er deutlich höhere CPU-Reserven, bessere I/O-Leistung, eine modernere Speicheranbindung und PCIe-Erweiterbarkeit bietet [zotero-item-144]. Gerade bei Sensorfusion, Kamerabildverarbeitung und parallelen ROS2-Knoten sind diese Reserven entscheidend. Die gemessenen Benchmark-Unterschiede zeigen zudem, dass der Raspberry Pi 5 im Single-Core-Bereich deutlich leistungsfähiger ist als der Raspberry Pi 4 [zotero-item-146].

Raspberry Pi OS Lite 64 Bit wird gewählt, weil es eine geringe Systemlast mit sehr guter Hardwareintegration verbindet. Der Verzicht auf eine Desktop-Umgebung reduziert RAM-Verbrauch, Hintergrundprozesse und Energiebedarf. Dies ist für ein batteriebetriebenes autonomes Fahrzeug besonders wichtig **[zotero-item-149]**.

ROS2 wird als Softwarebasis gewählt, weil es eine modulare Architektur für Robotiksysteme ermöglicht und die Kommunikation zwischen Sensorik, Aktorik, Wahrnehmung und Steuerung strukturiert unterstützt **[zotero-item-156]**. Die native Installation ist für ein Einzelentwickler- oder Studienprojekt zunächst die effizienteste Lösung, da sie weniger Systemkomplexität verursacht und direkten Hardwarezugriff erleichtert.

Docker wird nicht als zwingender Bestandteil der Zielarchitektur festgelegt, sondern als optionale Ergänzung bewertet. Der Einsatz ist sinnvoll, wenn mehrere Entwickler beteiligt sind, mehrere ROS2-Komponenten sauber getrennt werden sollen oder reproduzierbare Deployments erforderlich sind. Wenn hingegen maximale Akkulaufzeit, maximale Performance oder ein möglichst einfaches System im Vordergrund stehen, ist eine native ROS2-Installation vorzuziehen. Diese differenzierte Entscheidung ist wichtig, da Docker zwar Wartbarkeit und Reproduzierbarkeit verbessert, aber zusätzlichen Administrationsaufwand und workloadequabhängigen Ressourcenbedarf verursacht **[gameessPerformanceEvaluationDocker2023]** **[dockerinc.RuntimeMetrics]**.

3.7 Fazit

Der Technikkorridor zeigt, dass der Raspberry Pi 5 mit Raspberry Pi OS Lite 64 Bit und ROS2 den besten technologischen Ausgangspunkt für ein autonomes Fahrzeug im Studien- oder Forschungsumfeld darstellt. Diese Kombination verbindet hohe Rechen- und I/O-Reserven mit geringer Systemlast, guter Hardwareunterstützung und hoher Zukunftssicherheit.

Der Raspberry Pi 4 bleibt für bestehende oder kostenkritische Projekte weiterhin nutzbar, ist für leistungsorientierte autonome Systeme jedoch weniger geeignet. Der Raspberry Pi 5 bietet durch seine stärkere CPU, den RP1-I/O-Controller, bessere Kamera- und USB-Anbindung sowie PCIe/NVMe-Erweiterbarkeit die deutlich tragfähigere Plattform **[zotero-item-144]** **[zotero-item-147]**.

Bei der Betriebssystemwahl ist Raspberry Pi OS Lite 64 Bit für den produktiven Fahrzeugeinsatz besonders geeignet, da es ressourcenschonend ist und eine sehr gu-

te Raspberry-Pi-Hardwareintegration bietet [zotero-item-149]. Ubuntu Server bleibt eine gute Alternative, wenn die Nähe zum ROS2- und Ubuntu-Ökosystem wichtiger ist als minimale Systemlast. Ubuntu Desktop ist vor allem als Entwicklungsplattform für Visualisierung, Simulation und Debugging sinnvoll.

Docker verbessert Wartbarkeit, Reproduzierbarkeit und Skalierbarkeit, sollte aber nicht pauschal als Pflichtbestandteil der Architektur betrachtet werden. Für Einzelentwicklungen und batteriebetriebene Studienprojekte ist Raspberry Pi OS Lite mit nativer ROS2-Installation meist die effizienteste Lösung. Docker lohnt sich insbesondere bei Teamprojekten, CI/CD-Prozessen oder mehreren verteilten Softwarekomponenten. Der empfohlene Technikkorridor lautet daher: Raspberry Pi 5, Raspberry Pi OS Lite 64 Bit, ROS2 nativ als Basis und Docker als optionale Erweiterung.

4 Autoumbau

5 Autonomes Fahren

5.1 Ausweichkurven

5.2 Herausforderungen

6 Implementierung Autonomes Fahren

6.1 Hinderniserkennung (Elena)

In einem ersten Ansatz zur Hinderniserkennung werden die vom Light Detection and Ranging (LiDAR)-Sensor bereitgestellten Messdaten direkt ausgewertet. Hierzu werden sämtliche Messpunkte innerhalb eines festgelegten Maximalabstands betrachtet. Befindet sich ein Messwert innerhalb dieses Abstandes, wird dieser unmittelbar als Hindernis interpretiert. Dieses Verfahren ermöglicht zwar grundsätzlich die Erkennung von Objekten vor dem Fahrzeug, erweist sich im praktischen Betrieb jedoch als wenig robust.

Insbesondere führen einzelne fehlerhafte Messpunkte bereits zu einer Hinderniserkennung. Solche Ausreißer entstehen beispielsweise durch Reflexionen, Messrauschen oder ungültige Distanzwerte. Das Fahrzeug beginnt dadurch teilweise ein Ausweichmanöver, obwohl sich kein tatsächliches Hindernis vor ihm befindet. Zusätzlich werden auch Objekte berücksichtigt, die sich außerhalb des relevanten Fahrbereichs befinden und somit keinen Einfluss auf die aktuelle Fahrspur haben.

Zur Verbesserung der Hinderniserkennung können mehrere Maßnahmen umgesetzt werden. Zunächst wird der ausgewertete Sichtbereich des LiDAR-Sensors auf einen Winkelbereich vor dem Fahrzeug begrenzt. Dadurch werden ausschließlich Messpunkte innerhalb des für die Fahrtrichtung relevanten Winkels berücksichtigt. Gleichzeitig wird ein maximaler Erkennungsabstand eingeführt, sodass weit entfernte Objekte die Fahrentscheidung nicht beeinflussen.

Ein weiterer wesentlicher Bestandteil ist die Clusterbildung. Hierbei wird ein Hindernis nicht mehr anhand eines einzelnen Messpunktes erkannt, sondern erst dann, wenn mehrere benachbarte Messpunkte räumlich zusammengehören. Einzelne Ausreißer werden dadurch zuverlässig unterdrückt, da sie keine ausreichende Anzahl benachbarter Messungen besitzen. Dieses Verfahren erhöht die Robustheit der Hinderniserkennung erheblich und reduziert Fehlentscheidungen.

Zusätzlich werden ungültige LiDAR-Messwerte verworfen. Hierzu zählen insbesondere unendliche oder außerhalb des gültigen Messbereichs liegende Werte. Erst nach dieser Vorverarbeitung werden die verbleibenden Messpunkte zur Hinderniserkennung verwendet. Die Kombination aus Sichtfeldbegrenzung, Distanzfilterung und Clusterbil-

dung bildet somit die Grundlage für eine zuverlässige Objekterkennung.

6.2 Gauß-basierte Hindernisvermeidung (Elena)

Nach der Erkennung eines Hindernisses besteht die Aufgabe, dieses Hindernis zu vermeiden. Dazu soll ein geeigneter Fahrpfad berechnet und daraus ein Lenkwinkel für das Fahrzeug abgeleitet werden. Statt eines konstanten Lenkwinkels soll hierfür eine Gaußfunktion verwendet werden, um ein ruhiges Fahrverhalten zu erzeugen und nach dem Ausweichmanöver wieder auf die Ursprüngliche Spur zurück zu gelangen.

Zu Beginn eines Ausweichmanövers speichert der Controller die aktuelle Fahrzeugposition als Referenzspur (`reference_y`). Diese Referenz bildet den Ausgangspunkt für die Berechnung der Ausweichspur. Anschließend wird die Position des Hindernisses aus den vom LiDAR gelieferten Winkel- und Entfernungsinformationen in kartesische Koordinaten umgerechnet. Auf Grundlage dieser Hindernisposition wird eine Gaußfunktion erzeugt, deren Amplitude den notwendigen seitlichen Sicherheitsabstand beschreibt. Die Gaußfunktion wird anschließend auf den Referenzpunkt des Fahrzeuges addiert und ergibt so eine kontinuierliche Sollspur.

Im Gegensatz zu einer vollständigen Spurplanung wird diese Ausweichspur jedoch nicht als fester Fahrplan bis zum Ende abgefahren. Stattdessen dient sie als lokale Orientierung für die Fahrzeugregelung. Während der Fahrt wird fortlaufend ein Lookahead-Punkt auf der berechneten Spur bestimmt. Aus der seitlichen Abweichung zwischen Fahrzeugposition und diesem Zielpunkt wird mithilfe einer geometrischen Beziehung der aktuell benötigte Lenkwinkel berechnet. Dadurch reagiert das Fahrzeug kontinuierlich auf den berechneten Fahrpfad, ohne für jeden Abschnitt einen festen Lenkwinkel vorzugeben.

Die Wahl einer Gaußfunktion bietet gegenüber einer konstanten Lenkstrategie mehrere Vorteile. Durch ihren stetigen Verlauf entstehen gleichmäßige Änderungen des Lenkwinkels, wodurch abrupte Richtungswechsel vermieden werden. Gleichzeitig kann die seitliche Auslenkung über die Parameter der Gaußfunktion an unterschiedliche Hindernisgrößen angepasst werden. Die Ausweichspur ist lokal begrenzt und beeinflusst daher nur den Bereich um das erkannte Hindernis.

Die aktuelle Implementierung stellt einen ersten funktionsfähigen Ansatz für eine Gauß-basierte Hindernisvermeidung dar. Bereits umgesetzt sind die Berechnung einer Gauß-

kurve, die Bestimmung eines Lenkwinkels anhand eines Lookahead-Punktes sowie die Speicherung der ursprünglichen Fahrspur als Referenz. Eine vollständige Spurverfolgung bis zur automatischen Rückkehr auf die Ausgangsspur ist jedoch noch nicht vollständig realisiert. Sobald im aktuellen System kein Hindernis mehr erkannt wird, wird der Hindernisvermeidungscontroller zurückgesetzt. Dadurch wird die berechnete Ausweichspur nicht bis zu ihrem Ende verfolgt. Die vorhandene Referenzspur bildet jedoch bereits die Grundlage für eine spätere Erweiterung, bei der das Fahrzeug nach dem Umfahren eines Hindernisses selbstständig auf seine ursprüngliche Fahrbahn zurückkehrt.

Damit bildet die aktuelle Implementierung die Grundlage für eine kontinuierliche Hindernisvermeidung. Gegenüber einer einfachen festen Lenkstrategie ermöglicht sie bereits ein deutlich ruhigeres Fahrverhalten und schafft gleichzeitig die Voraussetzungen für zukünftige Erweiterungen, wie eine vollständige Spurverfolgung oder die Berücksichtigung mehrerer gleichzeitig erkannter Hindernisse.

6.3 Optimierung der Spurplanung (Elena)

Während der ersten Implementierung wurde die Ausweichtrajektorie nach jedem eingehenden LiDAR-Scan vollständig neu berechnet. Da sich die gemessenen Hindernispositionen durch Messrauschen geringfügig ändern, entstanden kontinuierlich unterschiedliche Trajektorien.

Dieses permanente Replanning führte dazu, dass sich der berechnete Lenkwinkel ständig änderte. In einigen Situationen wechselte das Fahrzeug sogar mehrfach zwischen einer Ausweichbewegung nach links und nach rechts oder begann Kreisbewegungen auszuführen. Statt das Hindernis zuverlässig zu umfahren, steuerte das Fahrzeug teilweise erneut darauf zu.

Aus diesem Grund wurde die Anzahl der Neuplanungen deutlich reduziert. Nach Beginn eines Ausweichmanövers bleibt die ursprünglich berechnete Trajektorie bestehen. Eine neue Planung erfolgt nur dann, wenn sich die Hindernissituation wesentlich verändert hat oder ein zusätzliches Hindernis erkannt wird. Dadurch bleibt die geplante Fahrbahn während des gesamten Manövers stabil.

Zusätzlich wird der berechnete Lenkwinkel mittels exponentieller Glättung gefiltert. Hierbei wird der aktuelle Lenkwinkel nicht vollständig übernommen, sondern mit dem zuvor

berechneten Wert kombiniert. Für den verwendeten Glättungsfaktor $\alpha = 0,7$ ergibt sich ein Tiefpassverhalten, das schnelle Änderungen unterdrückt und kleine Schwankungen der LiDAR-Messung ausgleicht. Dadurch entstehen weichere Lenkbewegungen und ein deutlich ruhigeres Fahrverhalten. Die aktuelle Implementierung replant nur bei relevanten Änderungen der Hindernissituation und glättet den Lenkwinkel mittels exponentieller Mittelung ($\alpha = 0.7$).

6.4 Priorisierung der Fahrbefehle (Elena)

Während des Betriebs sendet die Smartphone-Anwendung kontinuierlich Fahrbefehle an den Fahrzeugcontroller. Gleichzeitig erzeugt die Hindernisvermeidung eigene Steuerbefehle, sobald ein Hindernis erkannt wird. Ohne zusätzliche Priorisierung würden beide Komponenten gleichzeitig Steuerbefehle senden.

In ersten Versuchen führte dies dazu, dass normale Fahrbefehle die berechneten Ausweichkommandos unmittelbar überschrieben. Das Fahrzeug begann zwar mit einem Ausweichmanöver, erhielt jedoch im nächsten Steuerzyklus erneut einen manuellen Fahrbefehl und kehrte sofort auf die ursprüngliche Fahrtrichtung zurück.

Zur Lösung dieses Problems besitzt der Befehl avoid innerhalb des Fahrzeugcontrollers eine höhere Priorität als normale Fahrbefehle (move). Während eines aktiven Ausweichmanövers werden eingehende Fahrbefehle der Smartphone-Anwendung ignoriert. Erst nach erfolgreichem Abschluss der Trajektorie wird wieder auf die manuellen Steuerbefehle reagiert. Diese Priorisierung sorgt dafür, dass ein begonnenes Ausweichmanöver zuverlässig abgeschlossen wird. Die Prioritätslogik wird im Car-Controller über einen Zeitstempel realisiert, der normale move-Kommandos während eines aktiven avoid-Manövers unterdrückt.

6.5 Ideen

6.6 Herausforderungen

Schnittstellen der Hindernisvermeidung

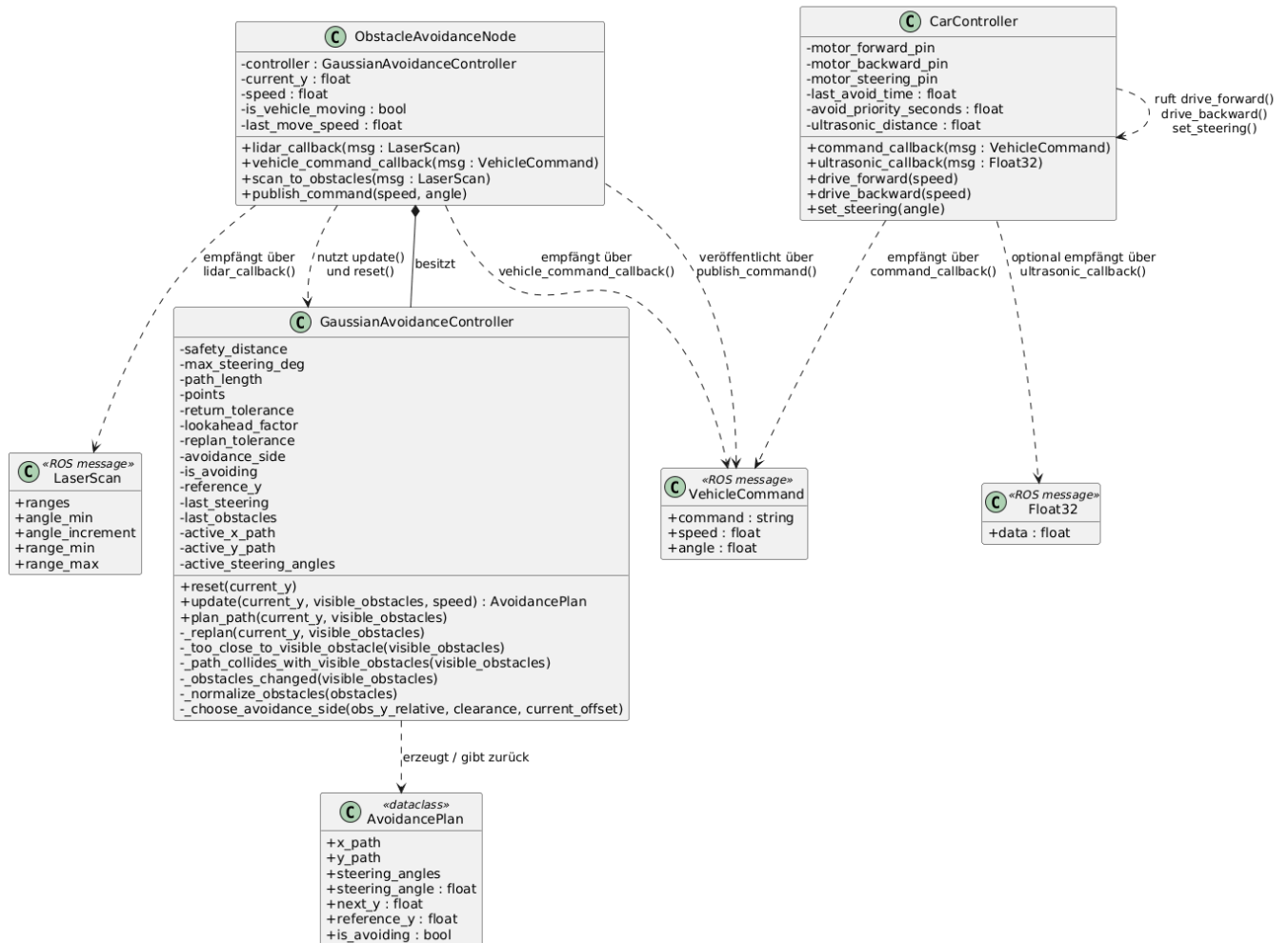


Abbildung 1: Klassendiagramm: Ausweichen

7 Aktualisierung der Applikation (Elena)

Um die Neigungssteuerung funktionsfähig und zuverlässiger zu gestalten, wurde die Sensoranbindung überarbeitet. Die ursprünglich verwendete Bibliothek `motion_sensors` wurde durch die modernere Bibliothek `sensors_plus` ersetzt. Dadurch werden nicht mehr die bereits berechneten Pitch- und Roll-Werte ausgegeben, sondern die Rohdaten des Sensors, aus denen die Neigungswinkel Pitch und Roll mathematisch berechnet werden. Dadurch ist die Steuerung nicht mehr von der fehleranfälligen Orientierungsfunktion der ursprünglichen Bibliothek abhängig und kann auf unterschiedlichen Geräten zuverlässiger arbeiten.

Die ermittelten Neigungswerte werden anschließend nicht mehr unmittelbar an das Fahrzeug übertragen, sondern zunächst verarbeitet. Hierfür wurde eine Deadzone eingeführt, die sehr kleine Bewegungen des Smartphones ignoriert. Dadurch werden un-

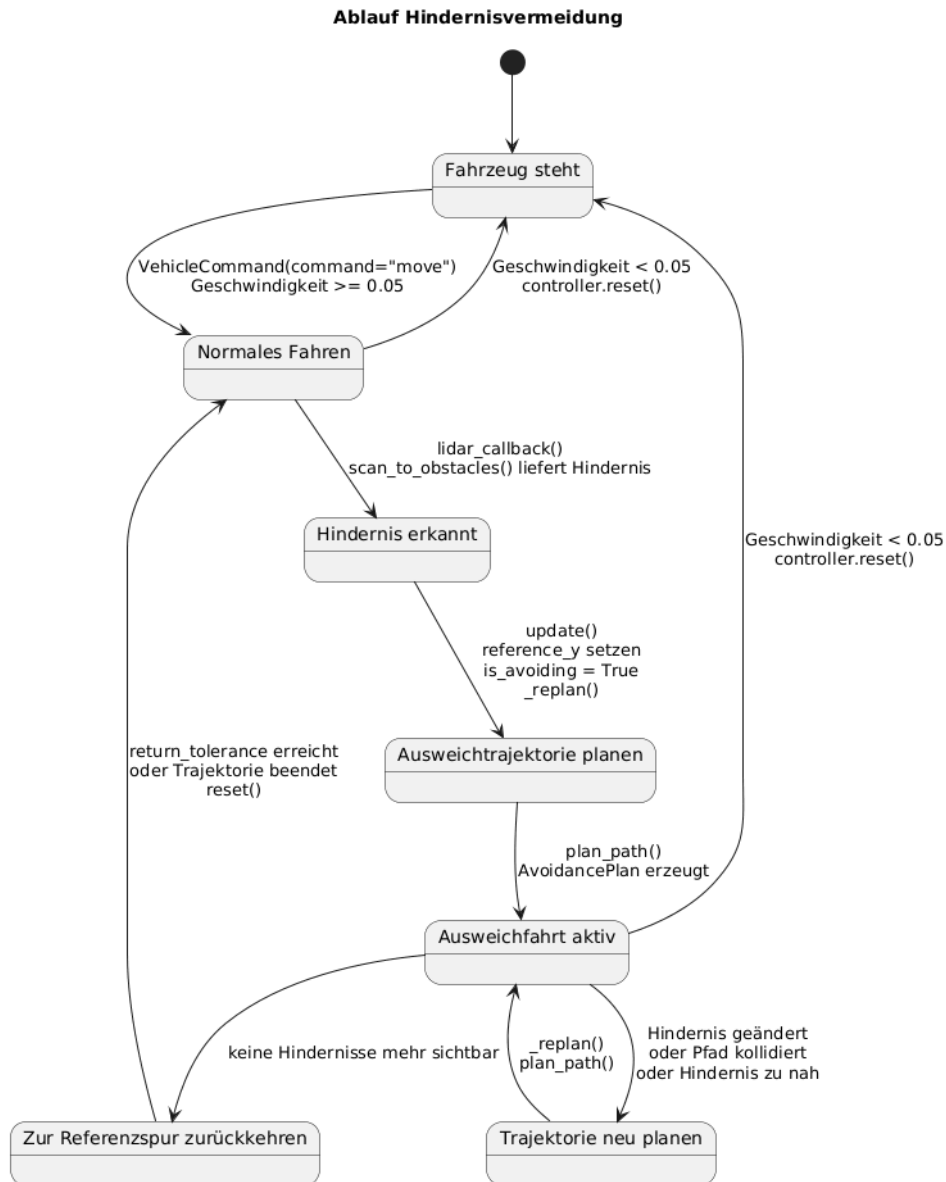


Abbildung 2: Zustandsdiagramm: Ausweichen

beabsichtigte Eingaben verhindert und das Fahrzeug bleibt bei ruhiger Haltung des Geräts stehen.

Zusätzlich wurde ein Low-Pass-Filter implementiert, der die Sensordaten glättet. Da Messwerte von Bewegungssensoren häufig Schwankungen und Rauschen enthalten, sorgt die Glättung für eine ruhigere und gleichmäßigere Fahrzeugsteuerung. Plötzliche kleine Ausschläge wirken sich dadurch nicht mehr unmittelbar auf das Fahrverhalten aus.

Um die Steuerung an unterschiedliche Fahrzeuge oder Nutzer anpassen zu können, wurden außerdem Verstärkungsfaktoren (Gain) für Geschwindigkeit und Lenkung eingeführt. Diese ermöglichen eine einfache Anpassung der Empfindlichkeit, ohne die ei-

gentliche Berechnung der Sensordaten verändern zu müssen. Ergänzend wurden Optionen zum Invertieren der Achsen integriert, sodass sich die Steuerungsrichtung bei Bedarf anpassen lässt.

Die berechneten Steuerwerte werden abschließend auf einen zulässigen Wertebereich begrenzt, bevor sie über den WebSocketManager an das Fahrzeug übertragen werden. Dadurch wird sichergestellt, dass nur gültige Werte an das Fahrzeug gesendet werden und unerwartete Sensorauschläge keine unzulässigen Steuerbefehle erzeugen.

Neben der eigentlichen Sensorverarbeitung wurden auch die Einstellungen innerhalb der App angepasst. Über einen Button kann nun ausgewählt werden, ob die Steuerung per Neigung oder per Joystick erfolgen soll. Die Neigungssteuerung wird nur dann geladen, wenn sie explizit aktiviert wurde. Dadurch unterstützt die Anwendung mehrere Steuerungsarten und bietet dem Nutzer mehr Flexibilität.

Darüber hinaus wurde die Verwaltung des Sensor-Streams verbessert. Beim Verlassen der Seite wird die Verbindung zum Sensor ordnungsgemäß beendet, wodurch unnötiger Ressourcenverbrauch und mögliche Fehler durch weiterhin laufende Sensorabfragen vermieden werden. Insgesamt führen diese Änderungen zu einer stabileren, besser anpassbaren und deutlich präziseren Neigungssteuerung.

8 Ergebnisse

8.1 Offene Herausforderungen

8.1.1 Grenzen der Hindernisvermeidung (Elena)

Die Hindernisvermeidung wurde so erweitert, dass sie ausschließlich während einer aktiven Fahrzeugbewegung arbeitet. Hierzu wird der aktuelle Bewegungszustand überwacht. Steht das Fahrzeug, wird die Hindernisvermeidung zurückgesetzt und es werden keine Ausweichtrajektorien berechnet. Dadurch wird verhindert, dass bereits im Stand unnötige Lenkbewegungen ausgeführt werden.

Eine weitere Verbesserung besteht in der automatischen Rückkehr auf die ursprüngliche Fahrspur. Zu Beginn eines Ausweichmanövers wird die seitliche Referenzposition gespeichert. Nach dem Passieren des Hindernisses führt die berechnete Gaußtrajektorie das Fahrzeug kontinuierlich auf diese Referenz zurück. Dadurch fährt das Fahrzeug nach dem Ausweichen wieder mittig auf seiner ursprünglichen Spur und verbleibt nicht dauerhaft versetzt.

Während der Entwicklung traten darüber hinaus sporadische Kommunikationsprobleme mit dem LiDAR-Sensor auf. Dabei erschien wiederholt die Fehlermeldung Descriptor length mismatch. Das Problem trat sowohl direkt nach dem Systemstart als auch während des Betriebs auf und ließ sich nicht zuverlässig reproduzieren. Im Rahmen der Fehlersuche wurden unter anderem Container-Neustarts, das erneute Verbinden der USB-Schnittstelle, das Leeren des seriellen Puffers, ein verzögerter Sensorstart sowie die Überprüfung der CPU-Auslastung und der Stromversorgung durchgeführt. Trotz dieser Maßnahmen konnte keine eindeutige Ursache identifiziert werden. Da der Fehler nur sporadisch auftritt, wird dieser Aspekt als bekannte Einschränkung des Systems dokumentiert.

Die größte funktionale Einschränkung betrifft derzeit die Auswahl der Ausweichseite. Die Entscheidung basiert ausschließlich auf den aktuell sichtbaren Hindernissen. Bei langen Hindernissen, Wänden oder teilweise verdeckten Objekten kann dadurch eine ungünstige Ausweichrichtung gewählt werden. Zukünftige Arbeiten könnten dieses Problem durch das Beibehalten einer einmal gewählten Ausweichseite, die Auswertung mehrerer aufeinanderfolgender LiDAR-Scans oder den Einsatz einer lokalen Occupancy-Grid- beziehungsweise Costmap-Repräsentation lösen.

8.2 Positives

9 Fazit und Ausblick

[dummyhidden]

Es wurden keine Literaturverweise im Dokument gesetzt.